# État d'avancement — ERP

Système de gestion pour compagnie de transport de voyageurs — 15 août 2026
Stack : Django + DRF (backend) · Angular standalone (frontend)
Multi-tenant (sociétés / agences) · i18n FR/EN

## 1. Vision & périmètre

ERP multi-sociétés (multi-tenant) pour une compagnie de transport : flotte, voyages, réservations/billetterie, colis & bagages, tarification, et administration (sociétés, agences, utilisateurs, rôles et permissions). Chaque société (institution) est isolée ; un super-administrateur plateforme provisionne les sociétés ; chaque société a un administrateur d'institution qui gère son personnel.
Modules métier à venir (specs fournies ultérieurement) : Trésorerie, Ressources Humaines (RH), Comptabilité.

## 2. Architecture des modules

### Backend — apps Django (apps/)

- core — socle (permissions, réponses, pagination, audit, devises).
- authentication — utilisateur custom, JWT, 2FA, sessions, me/, me/permissions.
- system_config — sociétés, agences, packages, abonnements, paramètres, feature-flags.
- admin_rbac — modules, menus, permissions, rôles, profils, affectations (cœur RBAC).
- users — gestion des utilisateurs (institution).
- fleet — flotte (véhicules, marques, modèles, conducteurs, carburant, maintenance, documents, missions, accidents, parkings, fournisseurs, contrats, config).
- voyages_trips — voyages (cycle de vie : soumission, départ, arrivée, embarquement, annulation, transbordement, équipages).
- voyages_booking — réservations, passagers, billets, retours, embarquement, feuille de route.
- voyages_config — config voyages (villes, points, trajets, plans de bus, catégories tarifaires, créneaux, colis, fidélité, modèle de billet, paramètres).
- voyages_consignment — colis & bagages (réception, manifeste, livraison, impressions).
- voyages_pricing — tarifs.
- hr — RH (modèles présents, vues minimales — à compléter).
- treasury — Trésorerie (modèles présents, vues minimales — à compléter).
- sync_scig_sage — synchronisation Sage / SCIG (inventaires, zones, transferts).
- online_booking — réservation en ligne (public).

### Frontend — Angular (angular/src)

Pages métier sous _api/pages/* (fleet, fleet-config, voyages, voyages-booking, voyages-config, voyages-consignment, voyages-pricing, admin, rbac, system, users, scig). Services sous _api/services/*. Menu latéral piloté par app/core/service/sidebar/sidebar.service.ts.

## 3. Travaux réalisés (session en cours)

### Corrections « mise en place d'une entreprise »

- Seed des devises (table Currency vide -> création d'entreprise échouait) : migration core/0003.
- Scoping société à la création d'agences, rôles et profils par un admin d'institution (isolation).
- Identité entreprise (RCCM, forme juridique, NIU, CNPS, TVA, capital) : affichée et éditable — ne s'écrase plus.

- me/ et login renvoient « company » comme objet (CompanyRef) et « is_superuser » -> corrige companyId nul et le gating du menu.
- Écrans multi-institutions réservés au super-admin (marqueurs de menu « super_admin »).

### Autres correctifs

- Création de bagage : tolérance des champs optionnels vides/null (serializer robuste + reset complet).
- Page d'attribution des permissions : hiérarchie Module -> Ressource -> cases d'action (CRUD) + sélection groupée.

## 4. RBAC — état détaillé (base pour la suite)

Le contrôle d'accès basé sur les rôles est complet et effectif de bout en bout. Section clé pour intégrer les futurs modules (Trésorerie, RH, Comptabilité).

### 4.1 Mécanisme

- Backend : classe core.permissions.HasModulePermission. Chaque vue déclare permission_classes = [IsAuthenticated, HasModulePermission] + un permission_map = { GET: <code>.view, POST/PATCH/PUT/DELETE: <code>.manage }.
- Autorisation : super-admin (is_superuser) -> tout ; admin d'institution (is_institution_admin) -> tout dans sa société (bypass) ; sinon vérifie le code via get_user_permissions(user, société).
- Frontend : directive standalone *hasPermission='code' (ou tableau, au moins un) ; masque menus et boutons ; réactive au chargement des permissions.
- Menu : filtrage générique dans sidebar.service.ts / buildMenuForUser — toute entrée avec « permission » est masquée si non accordée ; groupes/modules vides masqués ; bypass super/admin institution.
- me/permissions renvoie ['*'] pour super-admin ET admin d'institution ; sinon les codes des rôles/profils.

### 4.2 Taxonomie des codes

Format : <module>.<ressource>.<action>. 102 permissions seedées, reproductibles via migrations (admin_rbac 0002 à 0007). Préfixes de module de permission (différents des noms d'apps) :

- fleet.* (33) -> app fleet (+ fleet.config.* groupé).
- voyage.* (11) -> app voyages_trips (trip, crew, dashboard).
- ticketing.* (17) -> apps voyages_booking (booking/customer/ticket) ET voyages_consignment (parcel, luggage).
- transport.* (14) -> app voyages_config (city/route/fareclass/terminal + transport.config.*) ET voyages_pricing (transport.fare.*).
- scig.* (27) -> app sync_scig_sage.

### 4.3 Modules déjà protégés (backend + menu + boutons)

- Flotte (fleet) — fleet.* — 0 IsInstitutionAdmin restant.
- Voyages (voyages_trips) — voyage.*.
- Réservations (voyages_booking) — ticketing.*.
- Config Voyages (voyages_config) — transport.* + transport.config.*.
- Colis & Bagages (voyages_consignment) — ticketing.parcel.* / ticketing.luggage.*.
- Tarifs (voyages_pricing) — transport.fare.*.

### 4.4 Rôles & profils par défaut

À la création d'une société (seed_default_roles, appelé par _bootstrap_company), 6 rôles + 6 profils homonymes scopés à la société sont créés (idempotent) : Chef d'agence, Guichetier, Régulateur, Gestionnaire de flotte, Agent colis & bagages, Comptable (lecture). Backfill :

python manage.py seed_default_roles [--company <id>].

## 5. Recette : intégrer un NOUVEAU module au RBAC

Procédure reproductible (suivie pour les 6 modules). À appliquer pour Trésorerie, RH, Comptabilité :

- 1. Définir les codes : <module>.<ressource>.<action> (view/manage, ou create/update/delete ; code groupé <module>.config.* pour la référence).
- 2. Seeder ces codes via une data-migration admin_rbac (sur le modèle de 0003-0007) — indispensable pour base fraîche/tests/prod.
- 3. Backend : sur chaque vue, remplacer IsInstitutionAdmin par [IsAuthenticated, HasModulePermission] + permission_map. Objectif : 0 IsInstitutionAdmin.
- 4. Menu : ajouter « permission » sur chaque entrée du module dans sidebar.service.ts.
- 5. Boutons : importer HasPermissionDirective + poser *hasPermission (envelopper dans <ng-container> les boutons déjà sous *ngIf).
- 6. Tests pytest (patron _make_staff) : .view -> lecture OK / écriture 403 ; .manage -> écriture OK ; admin -> tout.
- 7. Rôles par défaut : ajouter les nouveaux codes aux bundles pertinents (admin_rbac/services/defaults.py) + rejouer seed_default_roles.

## 6. Prochaines étapes — Trésorerie, RH, Comptabilité

- Trésorerie (app treasury existante : modèles présents, vues minimales) — compléter les endpoints puis appliquer la recette RBAC (codes ex. treasury.*).
- RH (app hr existante : modèles présents, vues minimales) — idem (codes ex. hr.*), prévoir un rôle « Responsable RH ».
- Comptabilité (à créer) — nouvelle app + codes ex. accounting.*, rôle « Comptable » enrichi.
- Pour chacun : brainstorm -> spec -> plan -> exécution, puis intégration RBAC (section 5), puis MAJ des rôles par défaut.
- Interconnexions : Réservations/Colis (encaissements) -> Trésorerie ; Trésorerie/Paie -> Comptabilité ; RH (employés) <-> conducteurs/équipages.

## 7. Points d'attention / dette technique

- Après tout changement du payload utilisateur (is_superuser, company objet, permissions), les sessions ouvertes doivent se reconnecter (cache localStorage).
- Les codes de permission n'étaient dans aucune migration (dev uniquement) : corrigé (0003-0007). Toujours seeder par migration.
- Avertissement staticfiles (dossier static manquant) — bénin.
- Taxonomie de permission (voyage/ticketing/transport) != noms d'apps (voyages_*) : documenté en 4.2.
- Durcissement possible : guards de route Angular pour bloquer l'accès direct par URL.